FIAP · ENGENHARIA DE SOFTWARE · 2º ANO (FEV)
Global Solution 2026.2 · Desafio CitizenTech

DOMAIN DRIVEN DESIGN - JAVA

RotaSeca

API RESTful em Spring Boot com Oracle - alertas de alagamento

Equipe: Equipe RotaSeca · **Turma:** 2ESPx (preencher)
Integrantes (ordem alfabética)

Christian Schunck de Almeida	RM563850
Guilherme Vilela Perez	RM564422
Gustavo Panham Dourado	RM563904
Paulo Cesar de Govea Junior	RM566034
Thomas Jeferson Santana Wang	RM565104

Sao Paulo, novembro de 2026

Sumário

1. Especificação do projeto 3

2. Arquitetura e regras de negócio 4

3. Diagrama de classes 6

4. Protótipos de tela (front-end) 9

5. Tabela de endpoints 11

6. Diagrama Entidade-Relacionamento 14

7. Instruções para executar o projeto 15

1. Especificação do projeto

Descritivo da solução

A **RotaSeca** é uma CitizenTech para quem mora no Parque Novo Mundo e nos bairros vizinhos da Zona Norte de São Paulo, região que com frequência entra em estado de atenção e de alerta para alagamentos nos boletins do CGE. Sensores de nível d'água nos trechos que costumam alagar e relatos dos próprios moradores alimentam esta **API RESTful em Java com Spring Boot**. A API decide, por regras de domínio, quando emitir ou encerrar um alerta e para quem enviá-lo. Uma página web (servida pela própria aplicação) mostra a situação de cada trecho, recebe relatos e lista os pontos de apoio.

Justificativa

Hoje o morador só descobre que a rua alagou quando já está nela. Os alertas oficiais são por região inteira da cidade ("Zona Norte em estado de atenção") e não dizem *qual rua* evitar. Quem mais sofre são idosos, pessoas com deficiência, pais com carrinho de bebê e trabalhadores que voltam a pé ou de ônibus à noite. Um dos integrantes da equipe mora no bairro e convive com o problema.

Objetivos

- Representar o domínio do problema (bairros, pontos de risco, sensores, leituras, cidadãos, relatos, alertas, notificações e pontos de apoio) em uma aplicação Java organizada em camadas e fácil de estender.
- Implementar a **capacidade central**: transformar uma medição de água (sensor ou relato confirmado) em alerta no nível certo e notificar os moradores do bairro pelo canal adequado ao perfil.
- Oferecer CRUD completo (GET, POST, PUT e DELETE) de todas as entidades, com validação, regras de negócio e respostas de erro padronizadas.
- Persistir os dados no Oracle da FIAP e publicar a aplicação no Render.

2. Arquitetura e regras de negócio

O projeto segue as camadas trabalhadas em aula: **model → DAO → BO → controller**, com uma camada transversal de **tratamento de exceções**.

Pacote	Responsabilidade
`model`	9 classes que espelham as tabelas do Oracle e 7 enums com os domínios (nível de alerta, status, perfil...). `PontoRisco.classificar()` e as transições de `StatusOcorrencia` são regras do próprio domínio.
`dao`	Acesso ao banco com JDBC puro (`PreparedStatement`). `BaseDao` centraliza conexões, ids gerados por IDENTITY e a tradução de `SQLException` (violação de integridade vira HTTP 409).
`bo`	Regras de negócio e validações (campos obrigatórios, limites, e-mail único, FK existente, transições de status) antes de chamar o DAO.
`controller`	Endpoints REST. Recebem e devolvem JSON, sem regra de negócio.
`exception`	Exceções de domínio e `GlobalExceptionHandler` (`@RestControllerAdvice`), que devolve sempre o mesmo corpo de erro.
`dto` / `util` / `config`	Respostas compostas (situação do ponto, resultado da avaliação), validador, fuso de São Paulo e CORS.

Regra central: leitura de água → alerta → notificação (`AlertaBO.avaliarNivel`)

1. A altura é classificada pelas cotas do ponto: abaixo da cota de atenção = normal; a partir dela = **ATENCAO**; a partir do ponto médio entre as cotas = **ALERTA**; a partir da cota crítica = **EMERGENCIA**.
2. Se o nível é **mais grave** que o alerta ativo (ou não há alerta ativo), o alerta anterior é encerrado, um novo é emitido com mensagem padrão e todos os moradores do bairro que aceitam alertas são notificados (idosos por WhatsApp, quem não tem telefone por e-mail, os demais por push).
3. Se o nível é igual ou menos grave, o alerta atual é mantido. Isso evita mandar várias mensagens enquanto a água oscila.
4. Se a água volta para baixo da cota de atenção, os alertas ativos do ponto são encerrados.
5. Sensores em manutenção ou inativos não podem registrar leituras (HTTP 422). Um relato de morador só gera alerta depois de confirmado por um agente (`ABERTA → CONFIRMADA`). Status finais (RESOLVIDA, DESCARTADA) não mudam mais.

Tratamento de exceções

Exceção	HTTP	Quando
`DadosInvalidosException`	400	Campos obrigatórios, tamanhos, faixas, e-mail inválido. Devolve todos os erros de uma vez em `detalhes`.
JSON mal formado / parâmetro inválido	400	Corpo ilegível, enum inexistente, id não numérico.
`RecursoNaoEncontradoException`	404	Id inexistente.
`ConflitoException`	409	E-mail ou código de série duplicado, notificação repetida, exclusão de registro com dependentes.

`RegraNegocioException`	422	Sensor fora de operação, transição de status proibida, FK para registro inexistente, alerta já encerrado.
`PersistenciaException` / outras	500	Falha inesperada. Fica registrada no log, sem expor detalhes ao cliente.

```
{ "timestamp": "2026-02-10T17:40:12", "status": 400, "erro": "Bad Request",
  "mensagem": "Os dados enviados sao invalidos", "caminho": "/api/cidadaos",
  "detalhes": ["email nao e um e-mail valido"] }
```

3. Diagrama de classes

Os diagramas foram gerados a partir do código-fonte (atributos, métodos públicos e dependências injetadas pelos construtores), então refletem exatamente o código Java entregue.

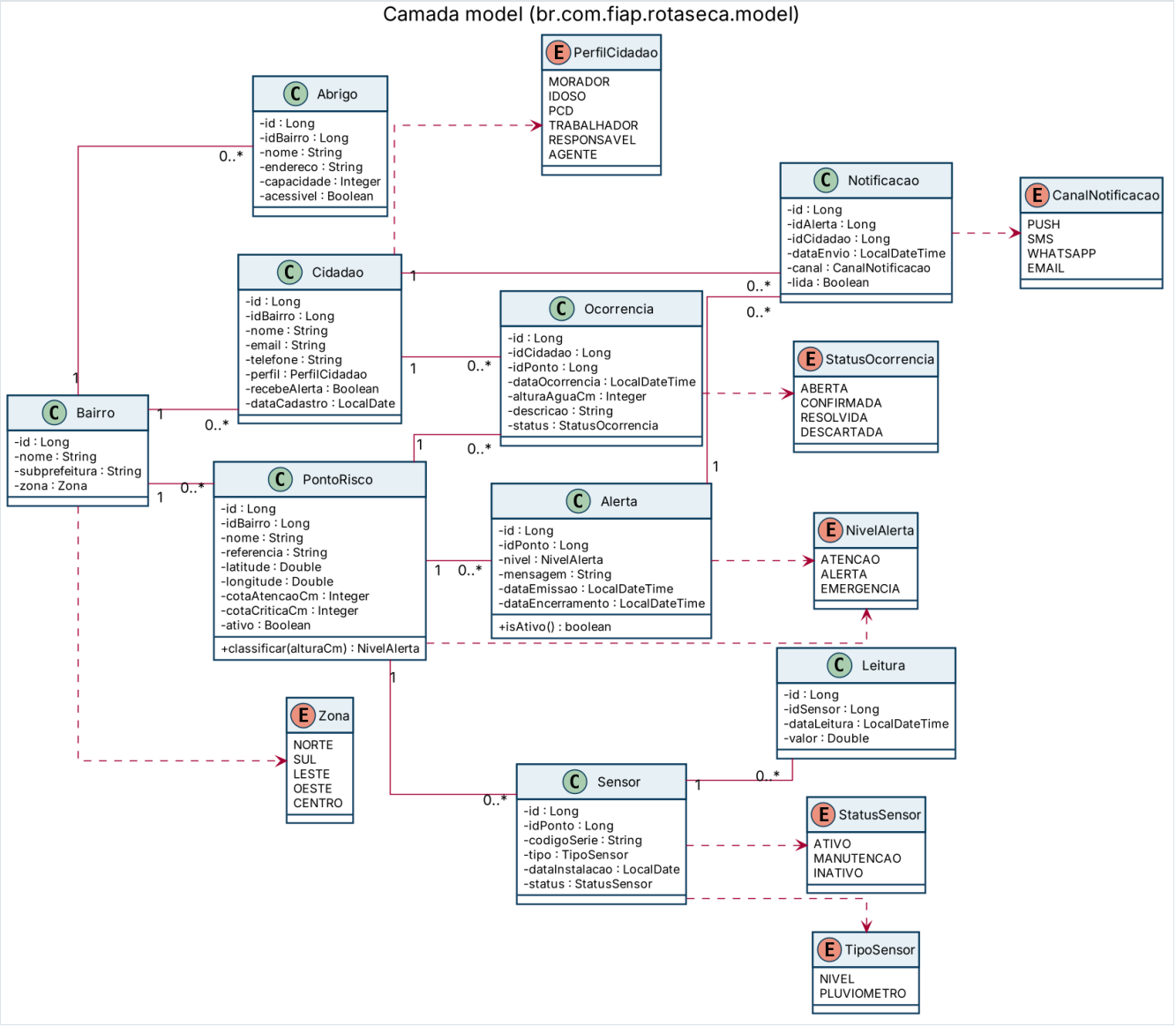


Figura 1 - Camada model: entidades, enums e relacionamentos (cardinalidades iguais às do banco)

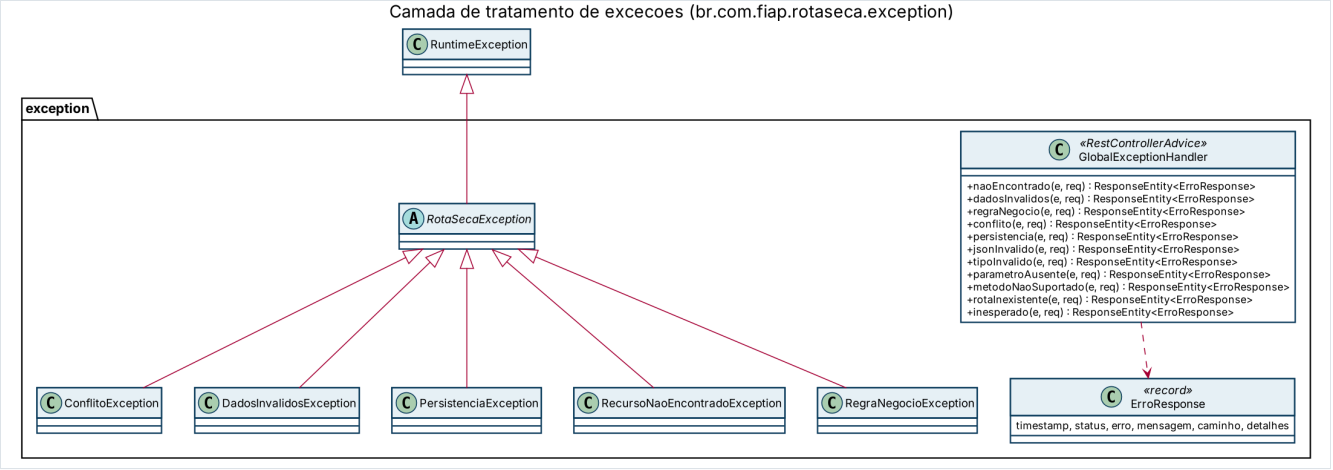


Figura 2 - Camada de tratamento de exceções

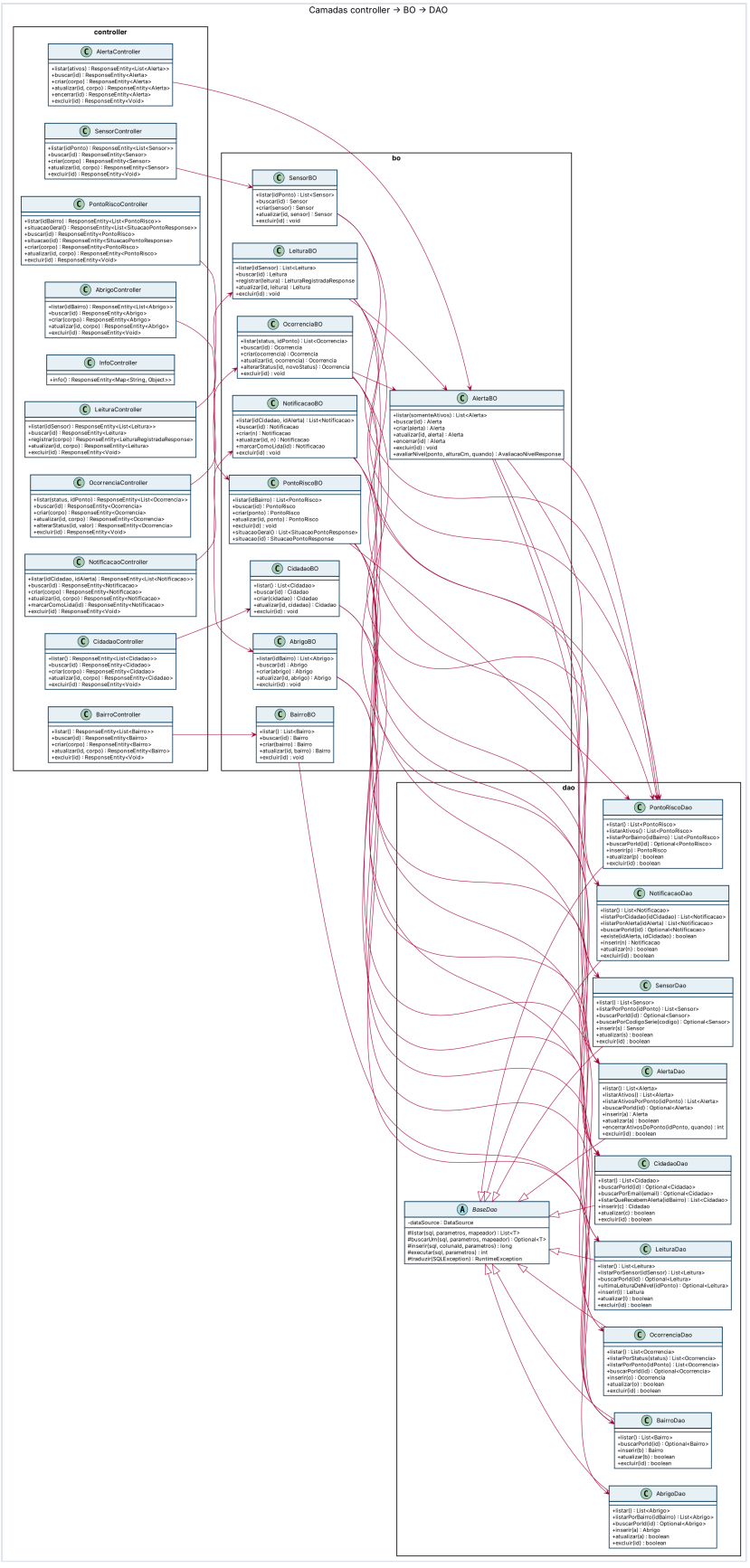


Figura 3 - Camadas controller → BO → DAO (setas = dependências injetadas; triângulo = herança de BaseDao)

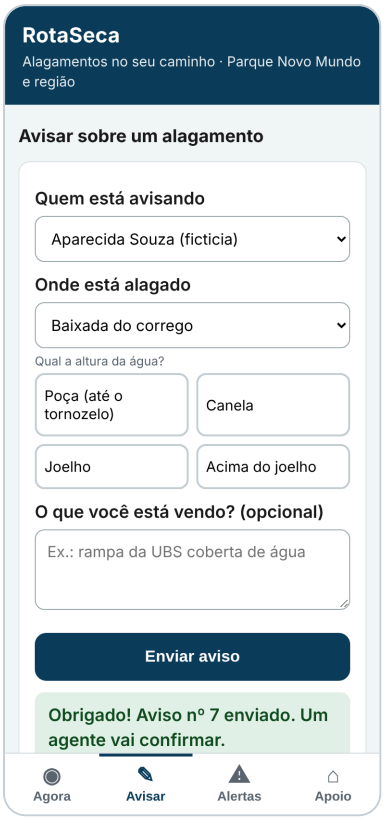
4. Protótipos de tela (front-end)

As telas abaixo são da página web incluída no projeto (`src/main/resources/static/index.html`), publicada na raiz do deploy. Ela foi pensada para celular, com letras grandes, alto contraste, linguagem simples e navegação inferior com 4 botões, porque o cidadão real não é especialista. As capturas usam o cenário simulado de um temporal.



Agora - situação das ruas

Consome `GET /api/pontos/situacao`. Os trechos aparecem do mais grave para o mais seguro. Cada cartão tem cor e texto (não só cor), a dica de ação ("Não atravesse. Use outro caminho."), a última medição do sensor e quantos vizinhos relataram o problema.



Avisar - relato de alagamento

Envia `POST /api/ocorrencias`. A altura da água é escolhida por referências do corpo (poça, canela, joelho) em vez de centímetros, o que é mais fácil para quem está na rua. O relato nasce com status ABERTA e é confirmado por um agente.



Alertas ativos

Consome `GET /api/alertas?ativos=true`. Mostra a mensagem gerada pela regra de negócio, o nível e o horário de emissão.



Pontos de apoio

Consome `GET /api/abrigos`. Mostra a capacidade e se o local é acessível para cadeira de rodas.

5. Tabela de endpoints

Base local: `http://localhost:8080` · Deploy: `https://rotaseca-api.onrender.com` · Documentação interativa: `/swagger-ui.html`. São 51 endpoints: 21 GET, 9 POST, 12 PUT e 9 DELETE.

Método	Endpoint	Descrição	Sucesso
GET	`/api/abrigos?idBairro=`	Lista os registros	200
GET	`/api/abrigos/{id}`	Busca pelo id	200
POST	`/api/abrigos`	Cadastra	201
PUT	`/api/abrigos/{id}`	Atualiza pelo id	200
DELETE	`/api/abrigos/{id}`	Exclui pelo id	204
GET	`/api/alertas?ativos=true`	Lista os registros	200
GET	`/api/alertas/{id}`	Busca pelo id	200
POST	`/api/alertas`	Cadastra	201
PUT	`/api/alertas/{id}`	Atualiza pelo id	200
PUT	`/api/alertas/{id}/encerrar`	Encerra um alerta ativo	200
DELETE	`/api/alertas/{id}`	Exclui pelo id	204
GET	`/api/bairros`	Lista os registros	200
GET	`/api/bairros/{id}`	Busca pelo id	200
POST	`/api/bairros`	Cadastra	201
PUT	`/api/bairros/{id}`	Atualiza pelo id	200
DELETE	`/api/bairros/{id}`	Exclui pelo id	204
GET	`/api/cidadaos`	Lista os registros	200
GET	`/api/cidadaos/{id}`	Busca pelo id	200
POST	`/api/cidadaos`	Cadastra	201
PUT	`/api/cidadaos/{id}`	Atualiza pelo id	200
DELETE	`/api/cidadaos/{id}`	Exclui pelo id	204
GET	`/api`	Confirma que a API esta no ar e lista os recursos	200
GET	`/api/leituras?idSensor=`	Lista os registros	200
GET	`/api/leituras/{id}`	Busca pelo id	200
POST	`/api/leituras`	Registra a leitura do sensor e, se passar das cotas, emite alerta e notifica	201
PUT	`/api/leituras/{id}`	Atualiza pelo id	200
DELETE	`/api/leituras/{id}`	Exclui pelo id	204

GET	`/api/notificacoes?idCidadao=&idAlerta=`	Lista os registros	200
GET	`/api/notificacoes/{id}`	Busca pelo id	200
POST	`/api/notificacoes`	Cadastra	201
PUT	`/api/notificacoes/{id}`	Atualiza pelo id	200
PUT	`/api/notificacoes/{id}/lida`	Marca a notificacao como lida	200
DELETE	`/api/notificacoes/{id}`	Exclui pelo id	204
GET	`/api/ocorrencias?status=&idPonto=`	Lista os registros	200
GET	`/api/ocorrencias/{id}`	Busca pelo id	200
POST	`/api/ocorrencias`	Cadastra	201
PUT	`/api/ocorrencias/{id}`	Atualiza pelo id	200
PUT	`/api/ocorrencias/{id}/status`	Muda o status do relato (?valor=CONFIRMADA); confirmar gera alerta	200
DELETE	`/api/ocorrencias/{id}`	Exclui pelo id	204
GET	`/api/pontos?idBairro=`	Lista os registros	200
GET	`/api/pontos/situacao`	Situacao atual de todos os pontos ativos (tela inicial)	200
GET	`/api/pontos/{id}`	Busca pelo id	200
GET	`/api/pontos/{id}/situacao`	Situacao atual de um ponto (ultima leitura, alerta ativo, relatos abertos)	200
POST	`/api/pontos`	Cadastra	201
PUT	`/api/pontos/{id}`	Atualiza pelo id	200
DELETE	`/api/pontos/{id}`	Exclui pelo id	204
GET	`/api/sensores?idPonto=`	Lista os registros	200
GET	`/api/sensores/{id}`	Busca pelo id	200
POST	`/api/sensores`	Cadastra	201
PUT	`/api/sensores/{id}`	Atualiza pelo id	200
DELETE	`/api/sensores/{id}`	Exclui pelo id	204

Exemplos de requisição

```
POST /api/leituras            (sensor 3 = nível d'água da Passagem sob a rodovia, cotas 20/40
cm)
{ "idSensor": 3, "valor": 45.0 }
-> 201 { "leitura": { "id": 13, ... },
         "avaliacao": { "nivelCalculado": "EMERGENCIA", "alertaEmitido": { "id": 6, "idPonto":
2, ... },
                        "alertasEncerrados": 0, "notificacoesEnviadas": 3 } }

POST /api/ocorrencias
{ "idCidadao": 5, "idPonto": 6, "alturaAguaCm": 20, "descricao": "Rampa da UBS alagada" }   ->
201 (status ABERTA)

PUT /api/ocorrencias/7/status?valor=CONFIRMADA      -> 200 (gera alerta se passar das cotas)

PUT /api/cidadaos/2
{ "idBairro": 1, "nome": "Rafael Lima", "email": "rafael.lima@exemplo.com",
  "telefone": "(11) 90000-0002", "perfil": "TRABALHADOR", "recebeAlerta": true }   -> 200

DELETE /api/bairros/1     -> 409 (bairro possui pontos, abrigos e cidadãos vinculados)
```

6. Diagrama Entidade-Relacionamento

Tabelas do banco Oracle usadas pela API. É o mesmo modelo entregue em Database Design: os scripts `01_DDL` e `02_DML` criam e populam o banco, e as classes da camada model espelham essas tabelas.

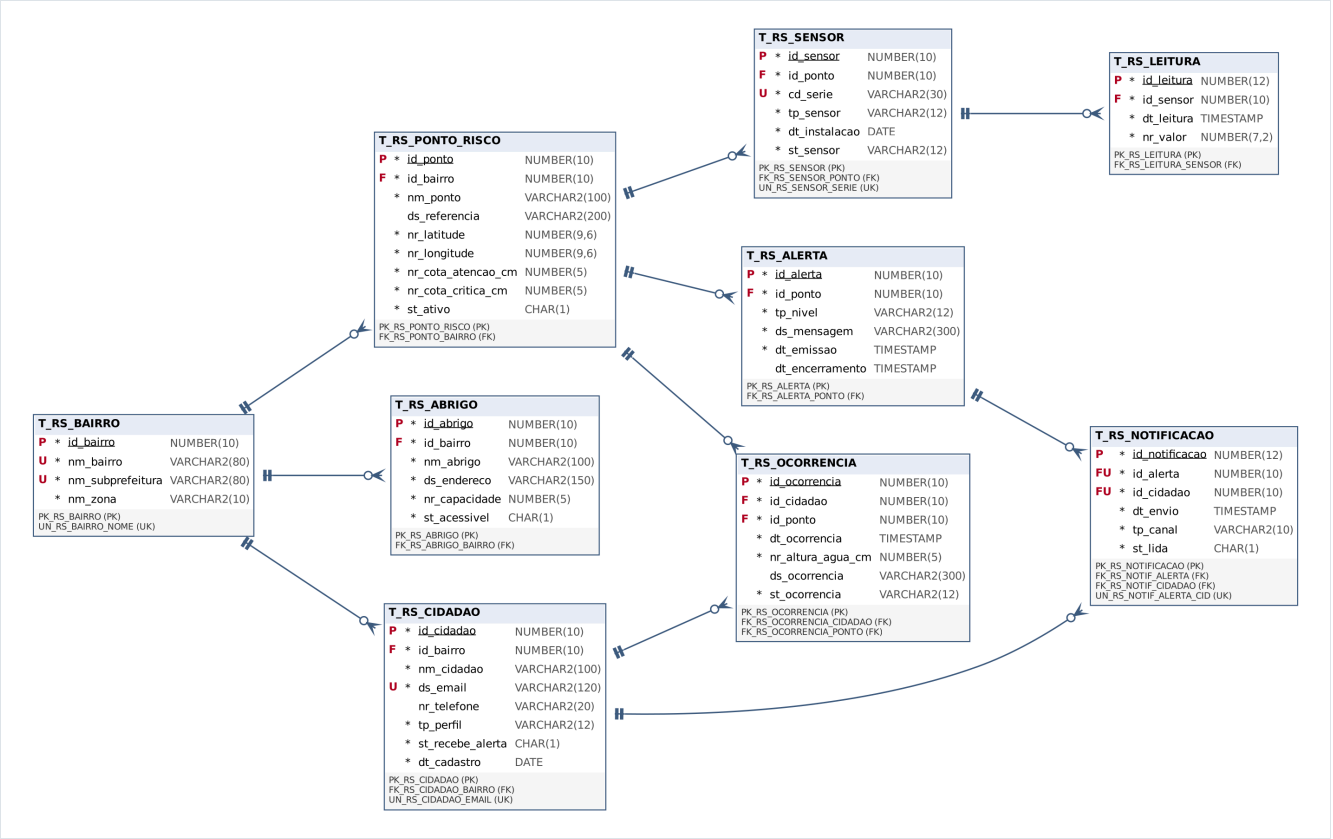


Figura 4 - DER (modelo relacional) com as 9 tabelas, chaves e cardinalidades

7. Instruções para executar o projeto

Ferramentas e versões

Item	Versão / uso
Java JDK	21
Spring Boot	3.5.6 (Web, JDBC/HikariCP), springdoc-openapi 2.8.9 (Swagger)
Maven	3.9+ (ou o Maven embutido do IntelliJ)
Banco de dados	Oracle Database da FIAP (`oracle.fiap.com.br:1521:ORCL`), driver ojdbc11. H2 em memória (modo Oracle) só para rodar localmente e nos testes.
IDE / ferramentas	IntelliJ IDEA, Oracle SQL Developer, Postman ou Swagger UI, Docker (deploy), Render.com

Links

- **GitHub:** https://github.com/SEU-USUARIO/rotaseca
- **Deploy (Render):** https://rotaseca-api.onrender.com (página web na raiz, API em `/api`, Swagger em `/swagger-ui.html`)

Passo a passo: rodar localmente

1. Clonar o repositório e abrir a pasta `java-api` no IntelliJ (ele reconhece o `pom.xml`).
2. Executar `RotaSecaApplication` (ou `mvn spring-boot:run`). Sem nenhuma configuração, sobe o perfil `dev` com H2 em memória já carregado com os mesmos dados do script DML.
3. Abrir `http://localhost:8080` (página web) e `http://localhost:8080/swagger-ui.html` (testar os endpoints).
4. Rodar os testes automatizados: `mvn test`.

Passo a passo: usar o Oracle da FIAP

1. No SQL Developer, conectar com o seu RM e executar `database/01_DDL_rotaseca.sql` e depois `02_DML_rotaseca.sql`.
2. Definir as variáveis de ambiente `SPRING_PROFILES_ACTIVE=prod`, `DB_USER=rmXXXXX` e `DB_PASSWORD=******` (opcional: `DB_URL`; o padrão é `jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL`).
3. Executar a aplicação. Ela passa a ler e gravar no Oracle.

Passo a passo: deploy no Render

1. Subir o repositório no GitHub.
2. No Render: *New* > *Web Service*, conectar o repositório, **Root Directory** = `java-api`, **Runtime** = Docker (o `Dockerfile` já está na pasta), plano Free. Também é possível usar o `render.yaml` da raiz como Blueprint.
3. Em *Environment*, cadastrar `SPRING_PROFILES_ACTIVE=prod`, `DB_USER` e `DB_PASSWORD`.
4. Fazer o deploy e testar `https://SEU-SERVICO.onrender.com/api`. Deve responder `"status": "UP"`.

> No plano gratuito o Render "dorme" depois de 15 minutos sem acesso, e a primeira requisição pode levar até 1 minuto. Abra o link antes da correção ou da apresentação.